

DEPLOYMENT GUIDE

West Africa Inputs Market Application (WAIMA)

Contract Reference: 7409372

Date: 13th August 2026

Submitted by



Step Innovations Africa Limited
P.O. Box 1088-00606
II Floor, Office Suites, Wing B,
Parklands Road, Nairobi, Kenya
Mobile: +254 710350839

Email: info@stepinnovationsafrica.com
Website: <https://stepinnovationsafrica.com>

Overview

Waima is a Laravel based web application developed using PHP and MySQL. This deployment guide provides the steps required to deploy and configure the application on a production server.

The guide covers application file deployment, web server configuration, environment configuration, directory permissions, database setup, and post deployment verification.

Server Requirements

The server should meet the following requirements:

- PHP $\geq$ 8.0
- MySQL $\geq$ 5.7 or MariaDB
- Apache / Nginx
- Composer (for dependency management)
- Enabled PHP Extensions:
 - OpenSSL
 - PDO
 - Mbstring
 - Tokenizer
 - XML
 - Ctype
 - JSON
 - BCMath
 - Fileinfo

Deployment Steps

Before beginning the deployment, ensure that the server meets the application's technical requirements and that the following are available:

- PHP with the version and extensions required by the Laravel application
- MySQL database
- A web server such as Apache or Nginx
- Composer, if application dependencies need to be installed on the server
- Access to the application source code and deployment files
- Database credentials
- Domain or subdomain configured to point to the server
- SSL certificate for HTTPS
- The production .env configuration file or the required environment variables

- The application database or the SQL database dump provided with the application

The exact PHP version and required PHP extensions should correspond to the version of Laravel and the application's composer.json configuration.

Deployment Configuration

The application configuration file is named .env. Update the following details with the actual production application URL and database credentials:

APP_DEBUG=false

APP_URL=https://yourdomain.com

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=your_database

DB_USERNAME=your_db_user

DB_PASSWORD=your_db_password

Deployment Steps

Upload Application Files

Upload the complete Waima application directory to the designated application directory on the server.

The Laravel application should retain its standard directory structure, including directories such as:

- app/
- bootstrap/
- config/
- database/
- public/
- resources/
- routes/
- storage/
- vendor/
- .env
- artisan
- composer.json

If the vendor/ directory is included in the deployment package, ensure that it is uploaded together with the application files.

If vendor/ is not included, install the application dependencies from the project root using Composer:

```
composer install --no-dev --optimize-autoloader
```

Do not place the entire Laravel project directory in the publicly accessible web root.

Configure the Web Server and Document Root

The domain or virtual host must point to the Laravel application's public directory.

For example: /var/www/waima/public

The public directory is the only application directory that should be directly accessible through the web server. This is important because the Laravel application contains sensitive files such as .env, configuration files, application source code, and database credentials that must not be publicly accessible.

For an Nginx deployment, the server configuration should route requests through Laravel's public/index.php.

For an Apache deployment, ensure that URL rewriting is enabled and that the Laravel public/.htaccess file is available.

Configure the Environment File

Create or update the .env file in the Laravel project root with the production environment settings.

Example:

```
APP_NAME=Waima
APP_ENV=production
APP_KEY=base64:YourAppKeyHere
APP_DEBUG=false
APP_URL=https://yourdomain.com
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=your_database
DB_USERNAME=your_db_user
```

DB_PASSWORD=your_db_password

The following values must be replaced with the actual production configuration:

- APP_KEY: The Laravel application encryption key.
- APP_URL: The application's production URL.
- DB_HOST: The database server hostname or IP address.
- DB_DATABASE: The name of the production database.
- DB_USERNAME: The database user with the required permissions.
- DB_PASSWORD: The corresponding database password.

APP_DEBUG must remain set to false in the production environment to prevent sensitive application and server information from being exposed to users.

If an application key has already been generated for the deployed application, retain the existing key. Do not generate a new key on an existing deployment unless the impact of changing encrypted application data has been assessed.

If a new application key is required, it can be generated from the application root using:
php artisan key:generate

Set Directory Permissions

Laravel requires write access to specific directories for logs, cached files, sessions, compiled files, and other runtime data. Set the appropriate permissions for:

```
chmod -R 775 storage
```

```
chmod -R 775 bootstrap/cache
```

Alternatively, where the web server runs under a dedicated user such as www-data, ownership can be assigned to the web server user:

```
chown -R www-data:www-data storage bootstrap/cache
```

The exact user and group should be confirmed based on the server's web server and PHP configuration. Avoid applying unrestricted permissions such as 777 to the application directories.

Configure the Database

Create the production MySQL database and database user, if they do not already exist. Then configure the corresponding credentials in the .env file. If a database SQL dump has been provided, import the SQL file into the configured database.

For example:

```
mysql -u your_db_user -p your_database < database.sql
```

Alternatively, if the application uses Laravel migrations and the database is intended to be created through migrations, run: `php artisan migrate --force`

The deployment team should use the database deployment method specified for the project. Do not run migrations against an existing production database without first confirming that the migration scripts are intended for that environment and that an appropriate database backup is available.

Create the Laravel Storage Link

If the application stores publicly accessible files using Laravel's storage system, create the symbolic link between the public directory and application storage:

```
php artisan storage:link
```

This creates the required link between:

`public/storage`

and:

`storage/app/public`

This step should be performed only if the application uses Laravel's public storage functionality.

Clear and Rebuild Application Cache

After configuring the production environment, clear any existing cached configuration and application data:

```
php artisan optimize:clear
```

For production deployments, the application cache can then be rebuilt:

```
php artisan optimize
```

This ensures that the application is using the current environment and configuration settings.

Verify Web Server and PHP Configuration

Confirm that the web server is running and that PHP is correctly configured to process the Laravel application.

Verify that:

- The domain resolves to the correct server.
- The document root points to the application's public directory.
- HTTPS is enabled.
- PHP is being processed correctly by the web server.
- Laravel URL rewriting is functioning correctly.

- The application can establish a connection to the MySQL database.
- The storage and bootstrap/cache directories are writable.
- The .env file is not publicly accessible.